

END-TO-END PROJECT: CREDIT CARD FRAUD DETECTION SYSTEM

Course: Data Science and Machine Learning

Project Type: Full End-to-End Production System

Environment: Kaggle (Training), VS Code (Application Development)

Backend: FastAPI (Port 8007)

Frontend: Streamlit

Model Serialization: Joblib or Pickle

Dataset: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

1. PROJECT OVERVIEW

The objective of this project is to build a complete, production-ready Credit Card Fraud Detection System from data exploration through model training to full deployment using an API and interactive frontend.

The student must:

1. Perform detailed exploratory data analysis.
2. Handle extreme class imbalance properly.
3. Train and compare multiple machine learning models.
4. Evaluate models using appropriate fraud-detection metrics.
5. Select and justify the best model.
6. Serialize the trained model.
7. Build a FastAPI backend running on port 8007.
8. Build a Streamlit frontend connected to the backend.
9. Ensure the application works end-to-end with real-time predictions.

This project simulates a real-world fintech fraud detection deployment pipeline.

2. DATASET DESCRIPTION

Dataset Name: Credit Card Fraud Detection

Source: Kaggle

File: creditcard.csv

The dataset contains credit card transactions made by European cardholders.

Features:

- Time: Seconds elapsed between this transaction and the first transaction.
- V1–V28: PCA-transformed numerical features.
- Amount: Transaction amount.
- Class: Target variable.
 - 0 = Legitimate transaction
 - 1 = Fraudulent transaction

Important Characteristics:

- Highly imbalanced dataset.
 - Fraud cases represent a very small percentage of total observations.
 - Accuracy is not an appropriate evaluation metric alone.
-

3. LEARNING OBJECTIVES

By completing this project, the student will:

1. Understand imbalanced classification problems.
2. Perform exploratory data analysis for anomaly detection.
3. Apply sampling techniques such as undersampling and SMOTE.
4. Train and compare multiple classification models.
5. Use fraud-relevant evaluation metrics.
6. Build and deploy a machine learning API.

7. Develop a frontend interface for ML inference.
8. Integrate backend and frontend into a working system.

4. PHASE 1: DATA EXPLORATION AND PREPROCESSING

This phase must be implemented in a Jupyter Notebook (.ipynb), preferably first developed on Kaggle.

Step 1: Load and Inspect Data

- Load dataset using pandas.
- Display first rows.
- Check shape and column types.
- Check for missing values.
- Display summary statistics.

Step 2: Exploratory Data Analysis

The student must include:

1. Class distribution plot.
2. Percentage of fraud vs non-fraud.
3. Correlation heatmap.
4. Distribution of Amount.
5. Distribution of Time.
6. Fraud vs non-fraud comparison for selected variables.
7. Boxplots for fraud vs non-fraud for Amount.

Step 3: Address Class Imbalance

The student must implement and compare at least two of the following:

- Random undersampling.
- Random oversampling.

- SMOTE (Synthetic Minority Oversampling Technique).
- Class-weight adjustment in models.

Explain the advantages and disadvantages of each approach.

Step 4: Feature Scaling

- StandardScaler must be applied to Amount and Time.
- Scaling must be saved as part of preprocessing pipeline.
- The preprocessing pipeline must be reusable for deployment.

Step 5: Train-Test Split

- Use stratified train-test split (e.g., 80/20).
- Maintain class distribution.

Deliverable for Phase 1:

fraud_training.ipynb containing:

- All EDA
- Preprocessing steps
- Sampling implementation
- Clear markdown explanations

5. PHASE 2: MODEL TRAINING AND SELECTION

The student must train at least three models:

Model 1: Logistic Regression

- Baseline model.
- Use `class_weight='balanced'`.

Model 2: Random Forest Classifier

- Suitable for nonlinear relationships.
- Evaluate feature importance.

Model 3: Gradient Boosting (e.g., XGBoost or sklearn GradientBoosting)

- Strong performance for imbalanced classification.

Optional Advanced Model:

- Isolation Forest (for anomaly detection comparison).

6. MODEL EVALUATION METRICS

Because this is a fraud detection system, evaluation must include:

1. Confusion Matrix
 - True Positive
 - False Positive
 - True Negative
 - False Negative
2. Precision
3. Recall
4. F1 Score
5. ROC-AUC Score
6. Precision-Recall Curve
7. PR-AUC

Important Note:

Recall is critical in fraud detection because missing fraudulent transactions has higher cost.

The student must:

- Present a comparison table of all models.
- Discuss trade-offs between precision and recall.

- Select the best performing model.
- Justify selection clearly.

7. MODEL SERIALIZATION

After selecting the final model:

- Save trained model using joblib or pickle.
- Save preprocessing objects (scaler).
- Ensure reproducibility.

Example saved files:

fraud_model.pkl

scaler.pkl

These files will be used by the FastAPI backend.

8. PHASE 3: FASTAPI BACKEND (PORT 8007)

The backend must:

- Load serialized model.
- Load scaler.
- Accept transaction input.
- Apply preprocessing.
- Return prediction and probability.

Project structure:

backend/

main.py

fraud_model.pkl

scaler.pkl

Required Endpoint:

POST /predict

Input JSON format:

```
{  
  "Time": 10000,  
  "V1": 0.1,  
  ...  
  "V28": -0.2,  
  "Amount": 150.75  
}
```

Output:

```
{  
  "prediction": 1,  
  "probability": 0.945,  
  "message": "Fraudulent transaction detected"  
}
```

Requirements:

- *Use Pydantic for validation.*
- *Proper error handling.*
- *Clear response structure.*
- *Run using:*

uvicorn main:app --reload --port 8007

Test using Swagger UI or Postman.

9. PHASE 4: STREAMLIT FRONTEND

The frontend must communicate with FastAPI backend.

Project structure:

frontend/

app.py

Frontend must include:

1. Manual Entry Mode
 - Input fields for all required features.
 - Submit button.
 - Display prediction and probability.
2. CSV Upload Mode
 - Upload CSV file.
 - Send rows to backend.
 - Display prediction results.
 - Highlight fraud rows.
 - Show fraud count summary.
3. Clean user interface.
4. Error handling for invalid inputs.

Communication:

Use requests library to send POST request to:

`http://localhost:8007/predict`

10. PHASE 5: END-TO-END TESTING

The student must demonstrate:

1. Backend running on port 8007.
2. Frontend running on Streamlit.
3. Manual prediction working.
4. Batch CSV prediction working.
5. Proper error handling.
6. Consistent predictions with notebook results.

11. COMPLETE FOLDER STRUCTURE

fraud_detection_system/

notebook/

fraud_training.ipynb

backend/

main.py

fraud_model.pkl

scaler.pkl

frontend/

app.py

requirements.txt

README.md

12. REQUIREMENTS FILE

The student must include:

pandas

numpy

scikit-learn

imbalanced-learn

xgboost (if used)

joblib

fastapi

uvicorn

streamlit

requests

matplotlib

seaborn

13. GRADING RUBRIC

Data Exploration and Preprocessing: 20%

Model Training and Evaluation: 30%

Model Selection Justification: 10%

FastAPI Backend: 20%

Streamlit Frontend: 10%

Documentation and Code Quality: 10%

14. FINAL SUBMISSION REQUIREMENTS

Student must submit:

1. Complete Jupyter Notebook.

2. Serialized model files.
3. FastAPI backend code.
4. Streamlit frontend code.
5. README with setup instructions.
6. Short technical report (1–3 pages) including:
 - Model comparison.
 - Final model justification.
 - Evaluation metrics.
 - Limitations.
 - Future improvements.

15. EXPECTED OUTCOME

At the end of this project, the student should be able to:

- Handle highly imbalanced datasets.
- Select models based on business impact.
- Use appropriate evaluation metrics.
- Build production-grade ML APIs.
- Deploy ML models for real-time prediction.
- Integrate backend and frontend systems.

Thank You.

Godspower Uyanga PhD

AI Specialist

godspoweruyanga3@gmail.com

<https://www.linkedin.com/in/godspower-uyanga-76aab01b0/>